

Link Prediction

Prof. O-Joun Lee

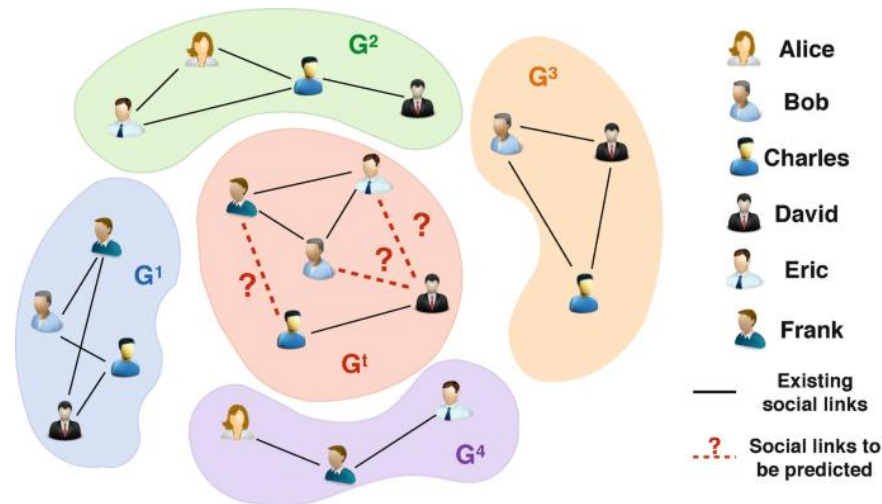
Dept. of Artificial Intelligence,
The Catholic University of Korea
ojlee@catholic.ac.kr

Contents

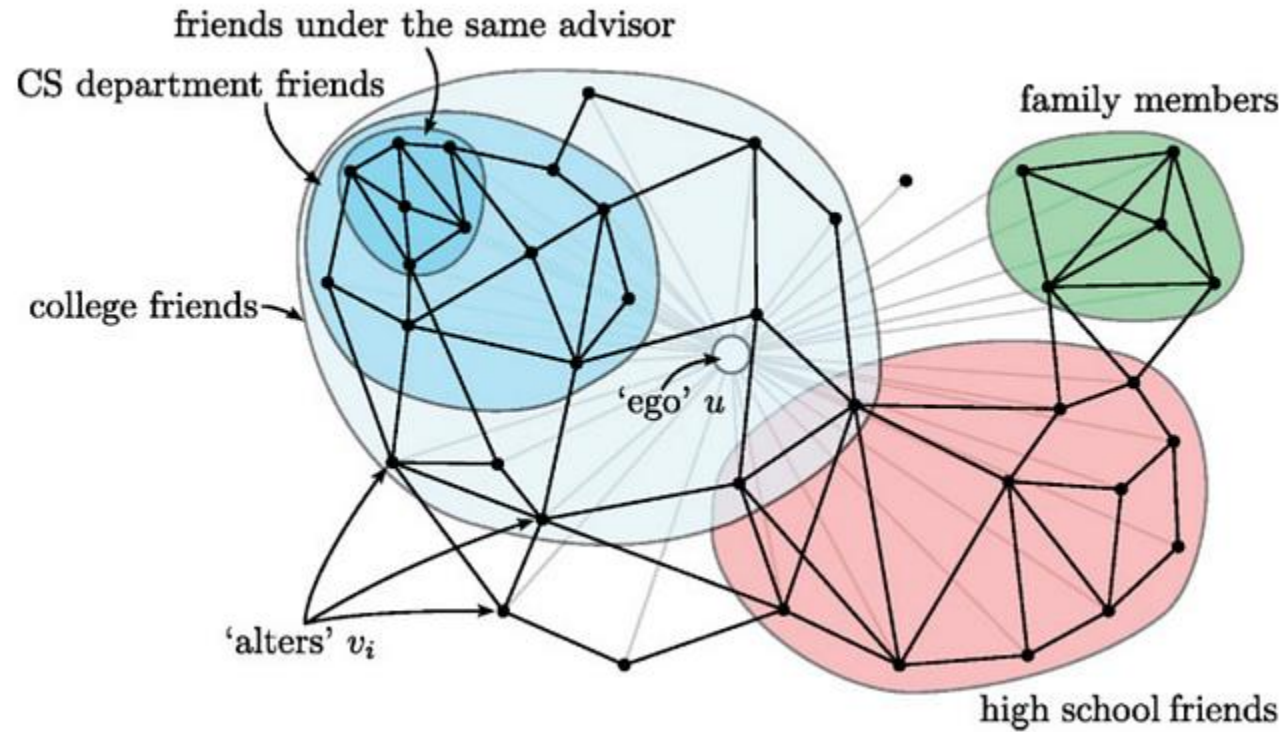


- Challenges in Link prediction
- Common methods
 - Common neighbours
 - Jaccard coefficient
 - Adamic-Adar
 - Preferential attachment
 - Path based
 - SimRank
- Tools for link prediction

- What we can do in social network prediction:
 - Friend Suggestions
 - Forecast likelihood of connection formation
 - Utilize network structure and historical interactions
 - Professional Networking: Suggesting co-authorship or collaboration opportunities in citation and academic networks.
 - User Behavior Prediction
 - Predict user engagement, churn, or sentiment
 - Inform marketing and product strategies

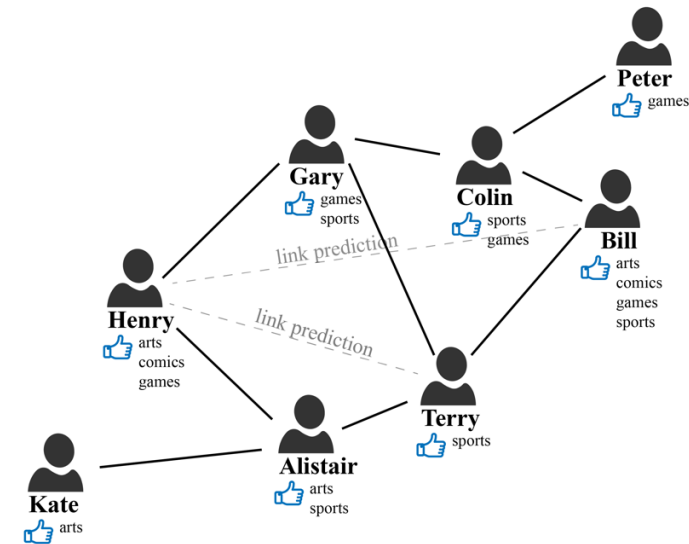
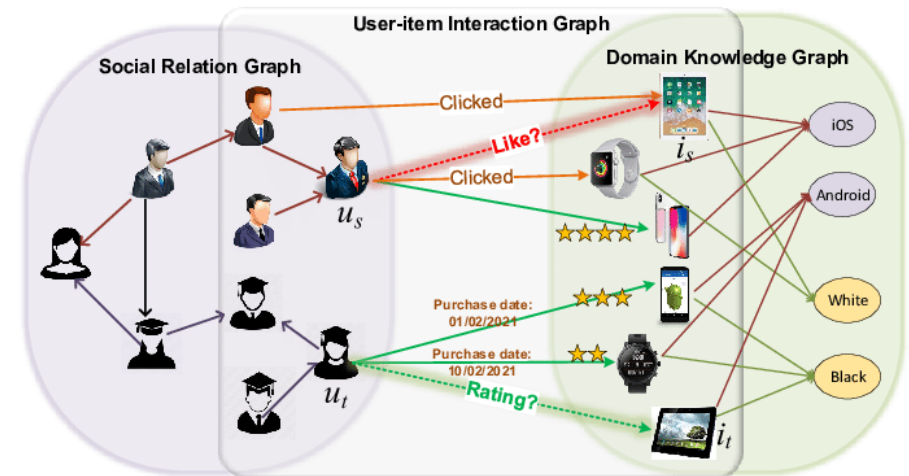


➤ Friend Suggestions task



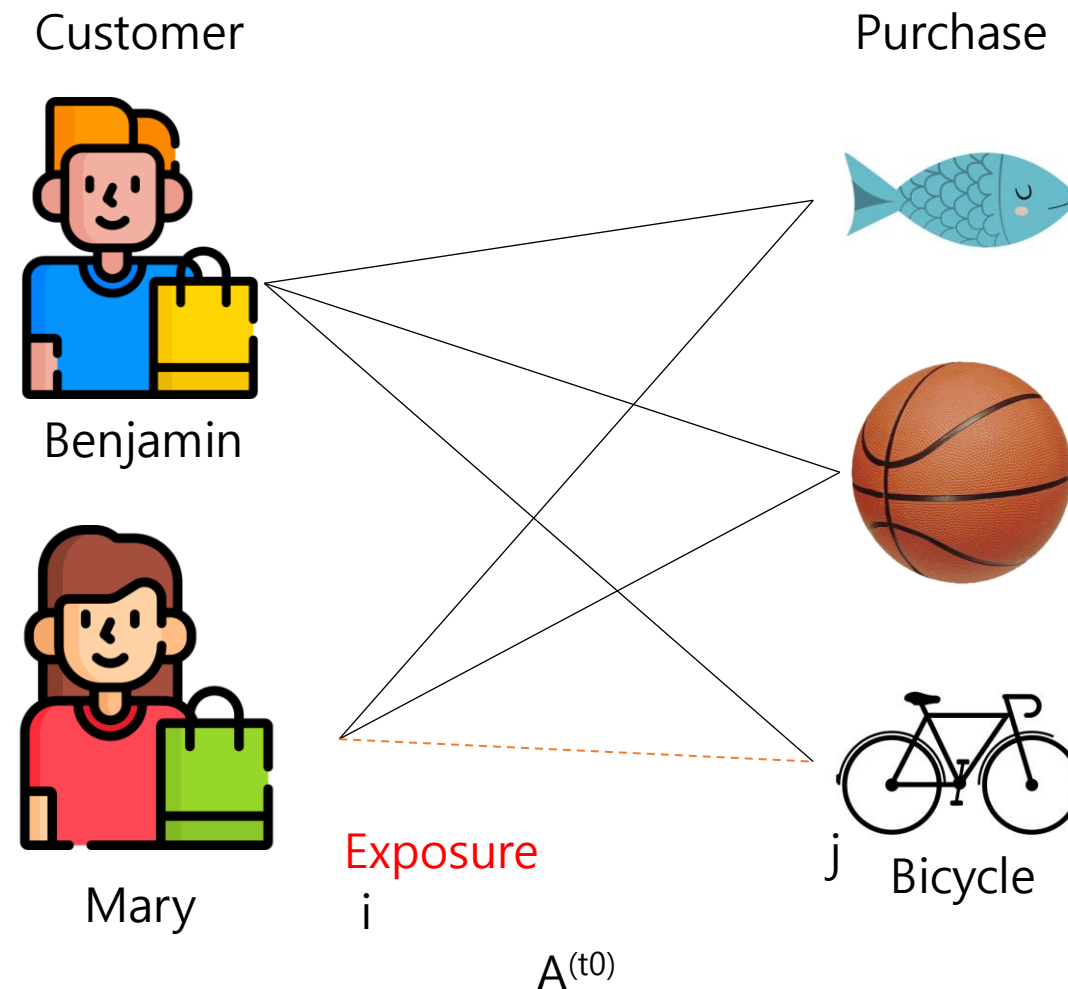
Facebook social circles where 'ego' is owner

- What we can do in recommendation systems prediction:
 - Rating Prediction
 - Predict user-item ratings
 - Ranking tasks: click / watch / purchase
 - Cold-Start Recommendation
 - Address cold-start problem for new users or items
 - Utilize content-based information or hybrid approaches
 - Implicit Feedback Prediction
 - Predict user interaction likelihood from implicit feedback



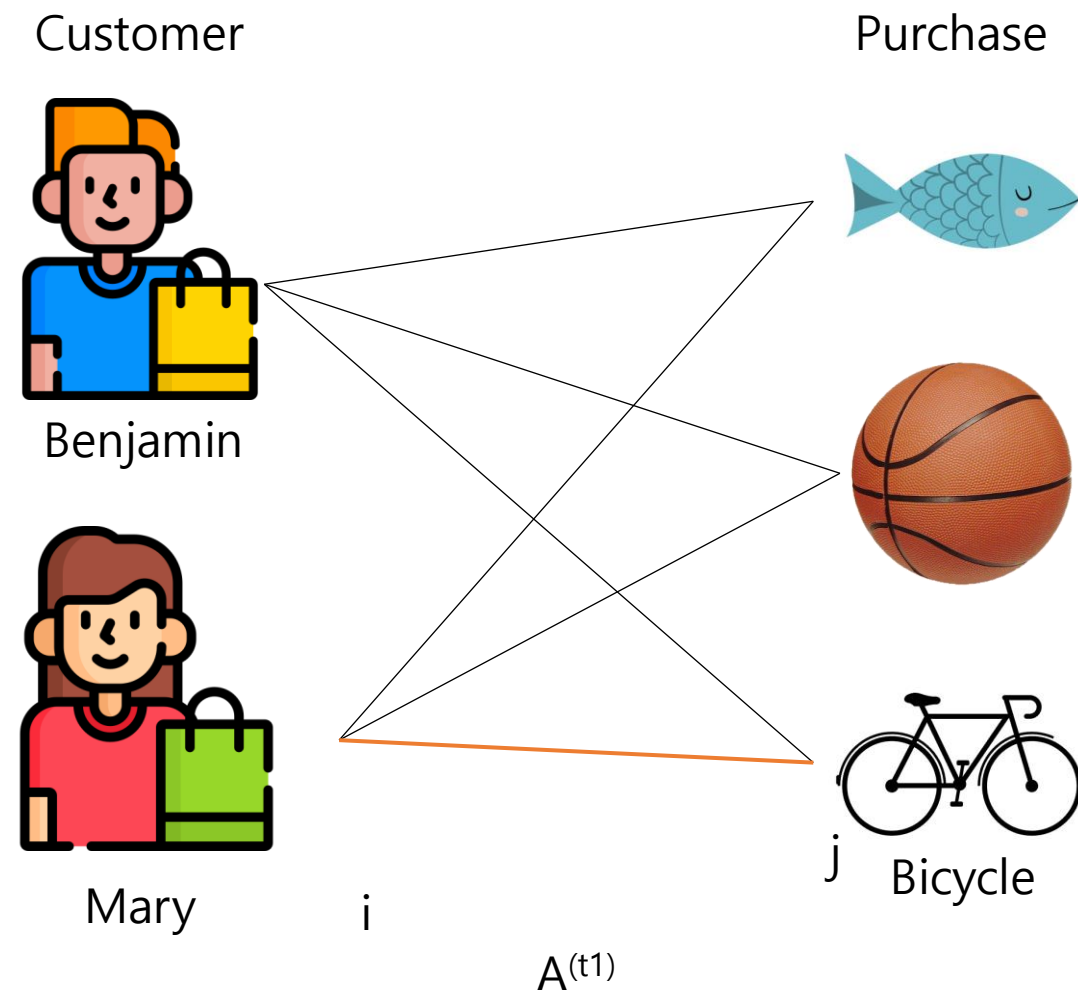
Link prediction as an exposure

- At time t_0 we expose Mary to Bicycle
- We will define this intervention exposure as
 - $E^{(t_0)} = (\text{Mary}, \text{Bicycle})$



Link prediction as an exposure

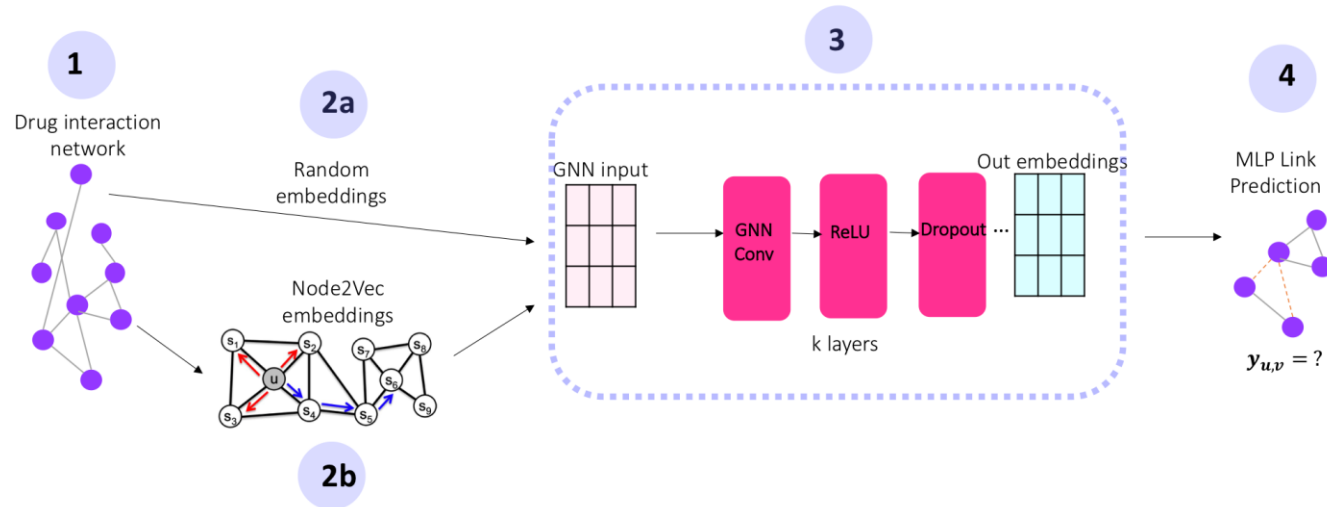
- At time t_1 we see if Mary bought Bicycle
- The outcome of the exposure at t_1
 - $A_{Mary, Bicycle}^{(t_1)} \in \{0, 1\}$



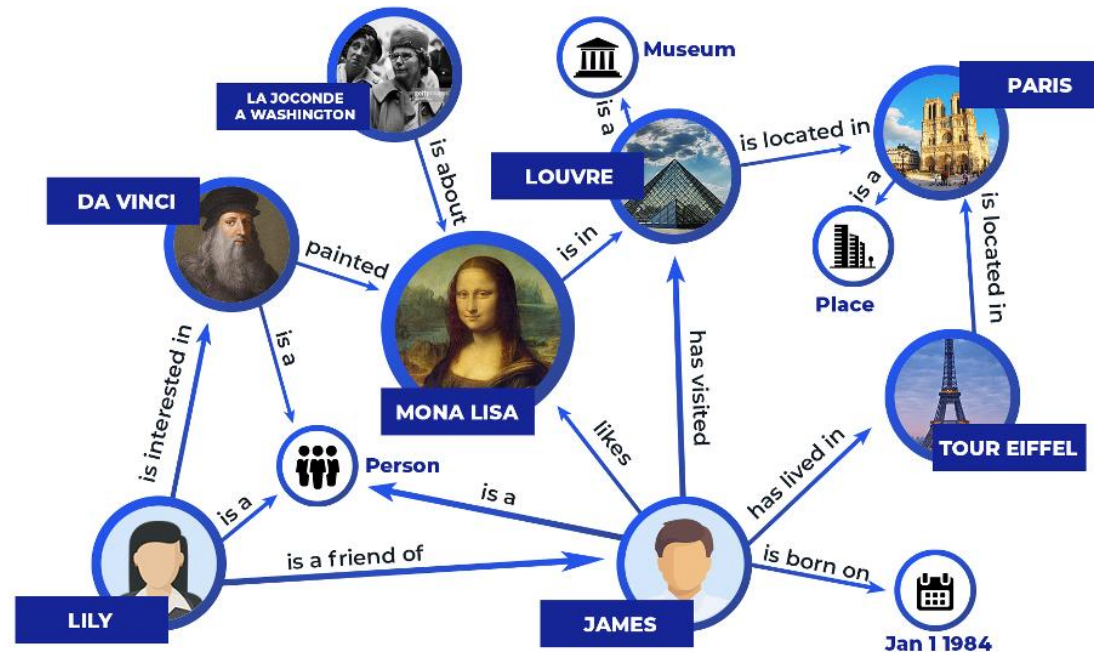
➤ What we can do in recommendation systems prediction:

- Bond prediction / bond completion
- Drug Discovery and Design
 - Identify drug candidates and optimize molecules
- Drug-drug, drug-target, or protein-protein interaction prediction

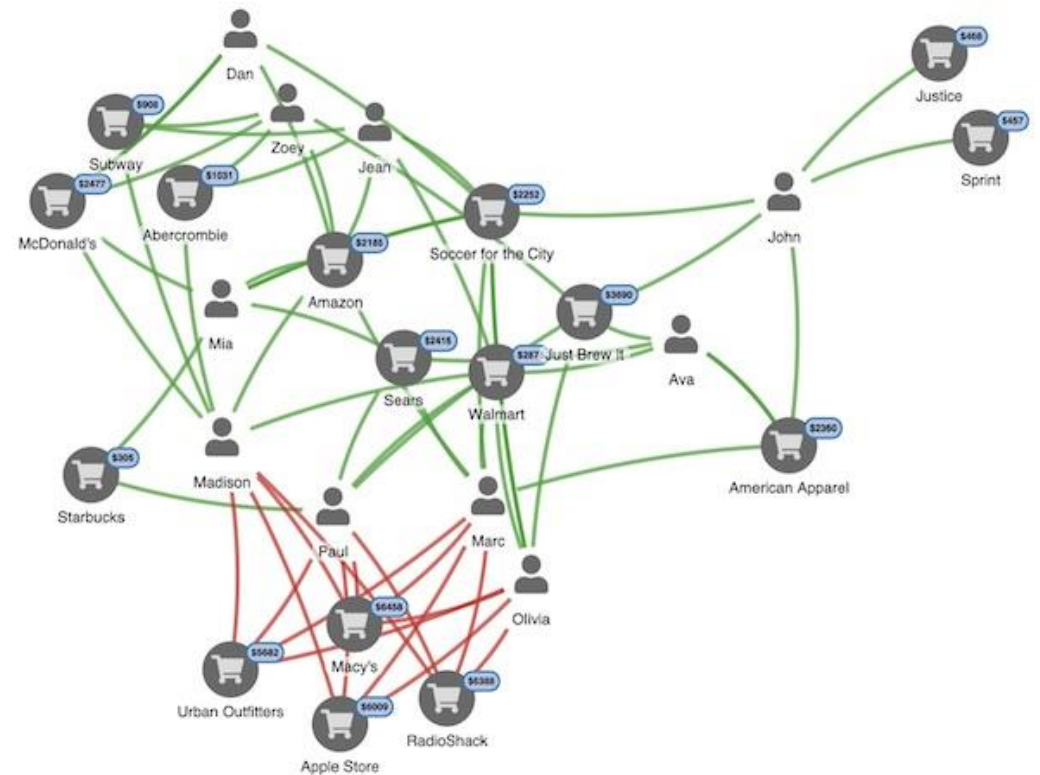
Note: Molecular property prediction is a graph prediction task, not Link Prediction



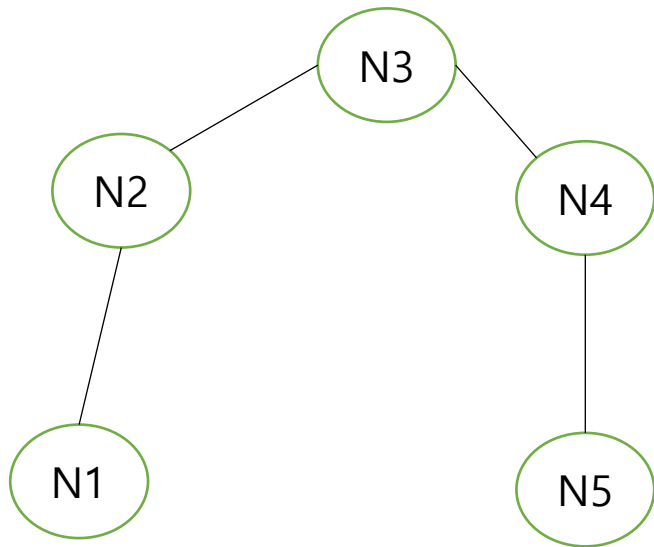
- What we can do in Knowledge & Data Management:
 - Knowledge Graph Completion: Inferring missing facts or relationships (e.g., in a dataset of historical figures or scientific facts) to create a more comprehensive database
 - Record Deduplication: Using link prediction to identify when multiple records actually refer to the same entity



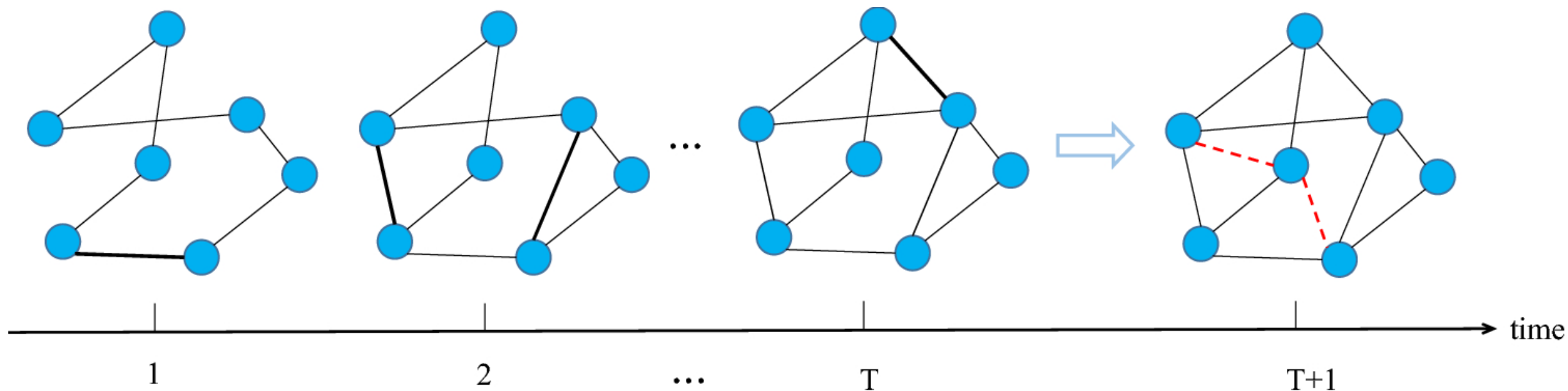
-
- Diagram illustrating a semantic network structure with entities and relationships:
- Entities:**
 - Crime Public order 31/08/2017** (Red circle icon)
 - Crime Vehicle crime 3/08/2017** (Red circle icon)
 - Craig Marshall** (Blue circle icon)
 - Anne Freeman** (Blue circle icon)
 - Billy Moore** (Blue circle icon)
 - Lillian Martinez** (Blue circle icon)
 - BL1 4BH 114 Grafton Street** (Green circle icon)
 - Relationships:**
 - Crime Public order 31/08/2017** *PARTY_TO* **Craig Marshall**
 - Crime Public order 31/08/2017** *OCCURRED_AT* **BL1 4BH 114 Grafton Street**
 - Crime Vehicle crime 3/08/2017** *PARTY_TO* **Billy Moore**
 - Crime Vehicle crime 3/08/2017** *OCCURRED_AT* **Lillian Martinez**
 - Craig Marshall** *KNOWS* **Anne Freeman**
 - Craig Marshall** *KNOWS_SN* **Anne Freeman**
 - Anne Freeman** *KNOWS* **Lillian Martinez**
 - Anne Freeman** *KNOWS_SN* **Lillian Martinez**
 - Lillian Martinez** *FAMILY_REL* **Billy Moore**
 - Lillian Martinez** *KNOWS* **Billy Moore**



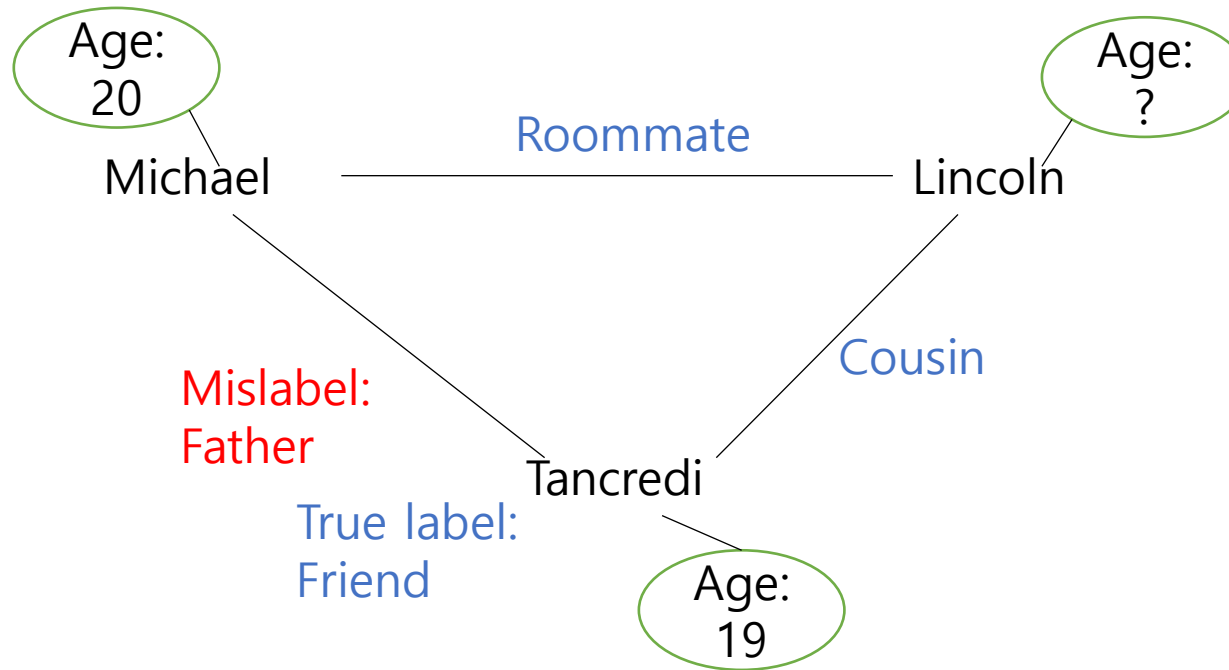
- Networks often exhibit sparsity, where the number of observed connections is much smaller than the total number of possible connections.
- This makes it challenging to infer missing links accurately, especially in large-scale networks
- A sparse graph has a number of edges closer to the minimal number of edges
- Eg: A graph has 5 nodes and only 4 edges



- Networks evolve over time, with new connections forming and existing connections breaking.
- Link prediction models must be able to capture and adapt to temporal dynamics, predicting future links based on past network states.



- Link prediction models often rely on node and edge attributes to infer connections
- However, real-world data may be incomplete or noisy, leading to challenges in feature selection and model training

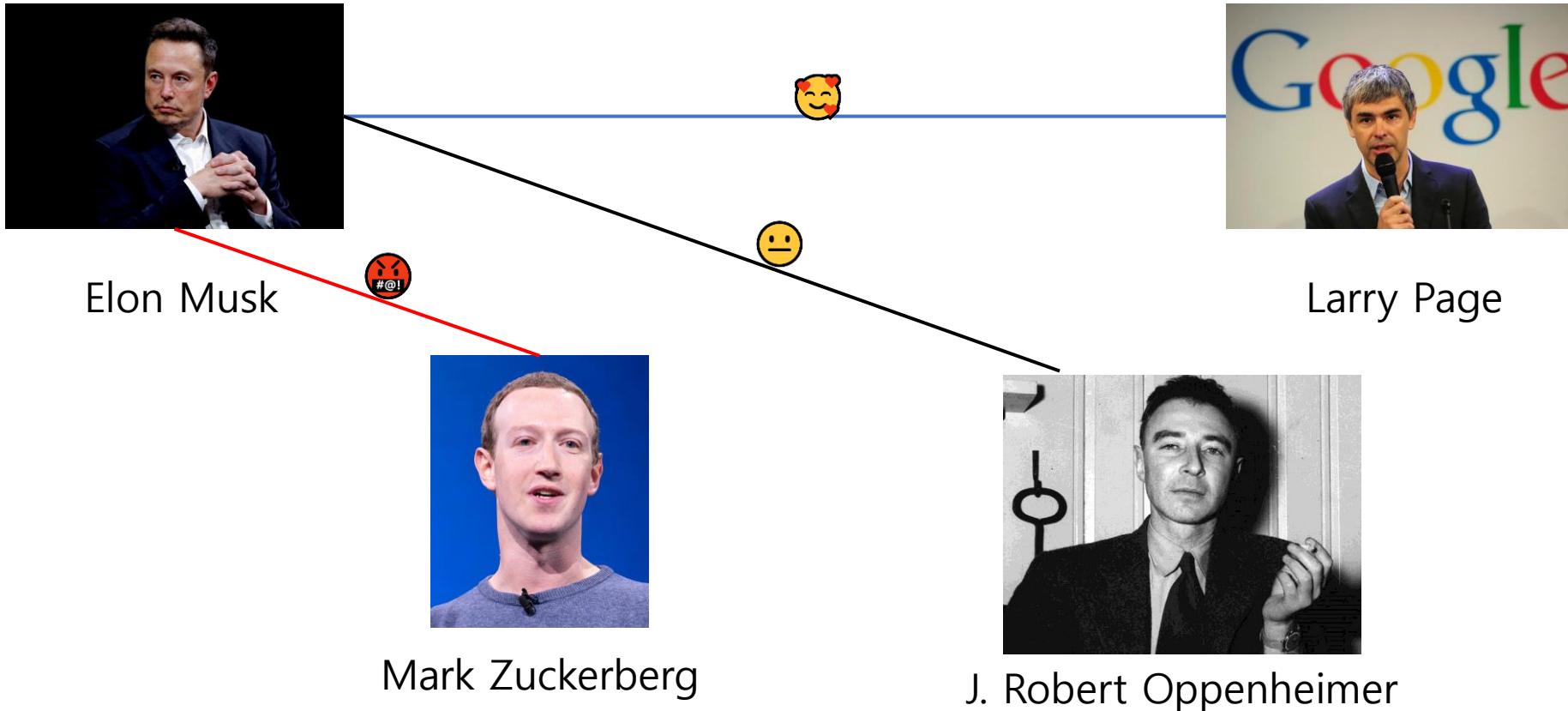


- There are 3 types of Link in Signed Networks
 - Positive Link
 - Positive links refer to connections or relationships that are expected or desirable in a network
 - These connections typically represent friendships, collaborations, interactions, or any other positive associations between nodes
 - Negative Link
 - Negative links represent relationships or connections that are undesirable or unlikely in a network
 - These connections might indicate conflicts, disagreements, competition, or any other negative associations between nodes
 - Neutral Link
 - Neutral links refer to connections that neither positively nor negatively affect the network
 - These connections are typically neither desirable nor undesirable; they might represent indifference or lack of significant interaction between nodes

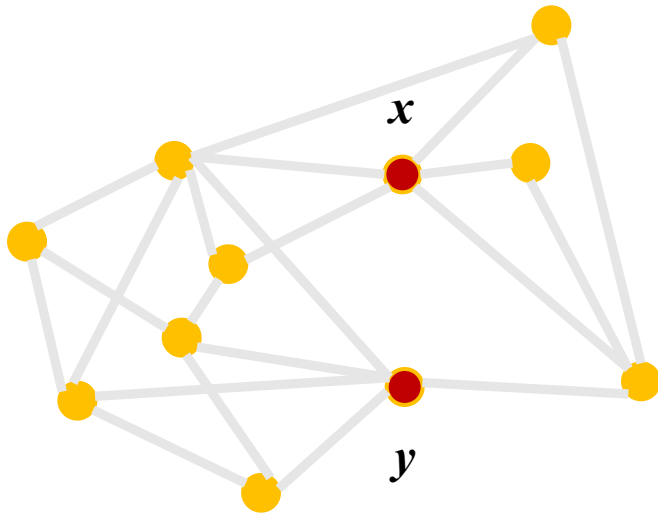
Source: Cai, S., Shan, W., & Zhang, M. (2022). Structure information learning for neutral links in signed network embedding. *Information Processing & Management*, 59(3), 102917.

➤ There are 3 types of Link in Signed Networks

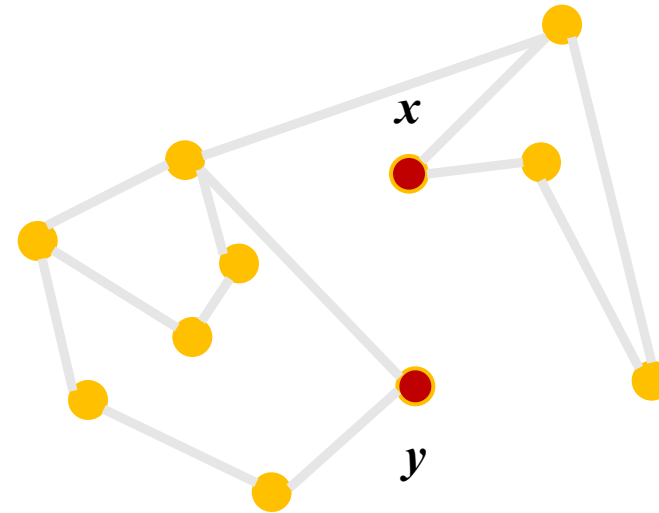
- Positive Link
- Negative Link
- Neutral Link



- Link prediction also equals to “Relation prediction” or “Graph completion” or “Relational inference” depending on the specific application domain
- In many networks, people who are “close” belong to the same social circles and will inevitably encounter one another and become linked themselves.
- Link prediction heuristics measure how “close” people are

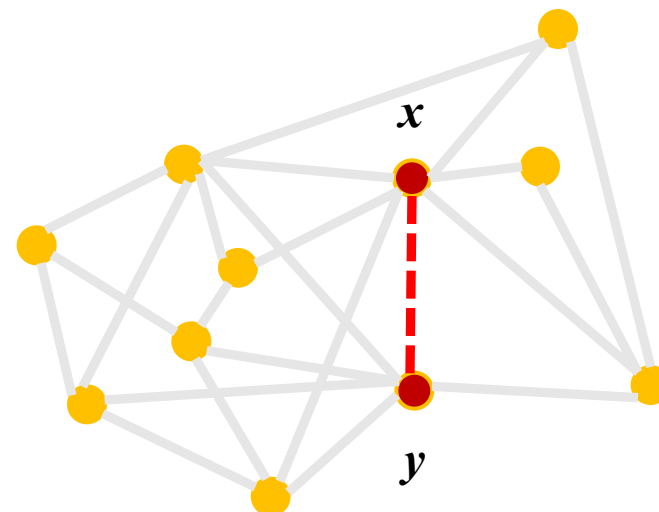


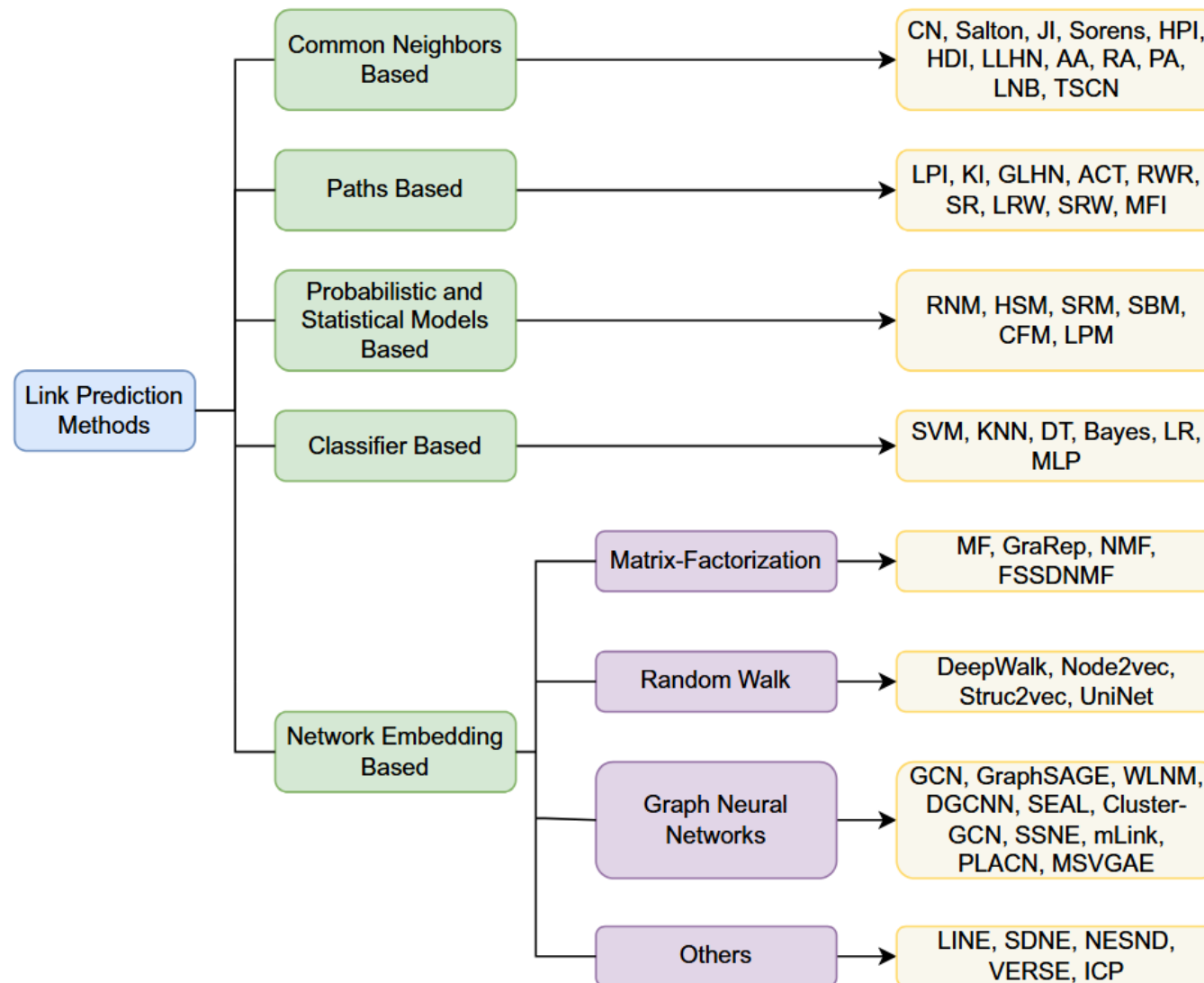
Red nodes are close to each other



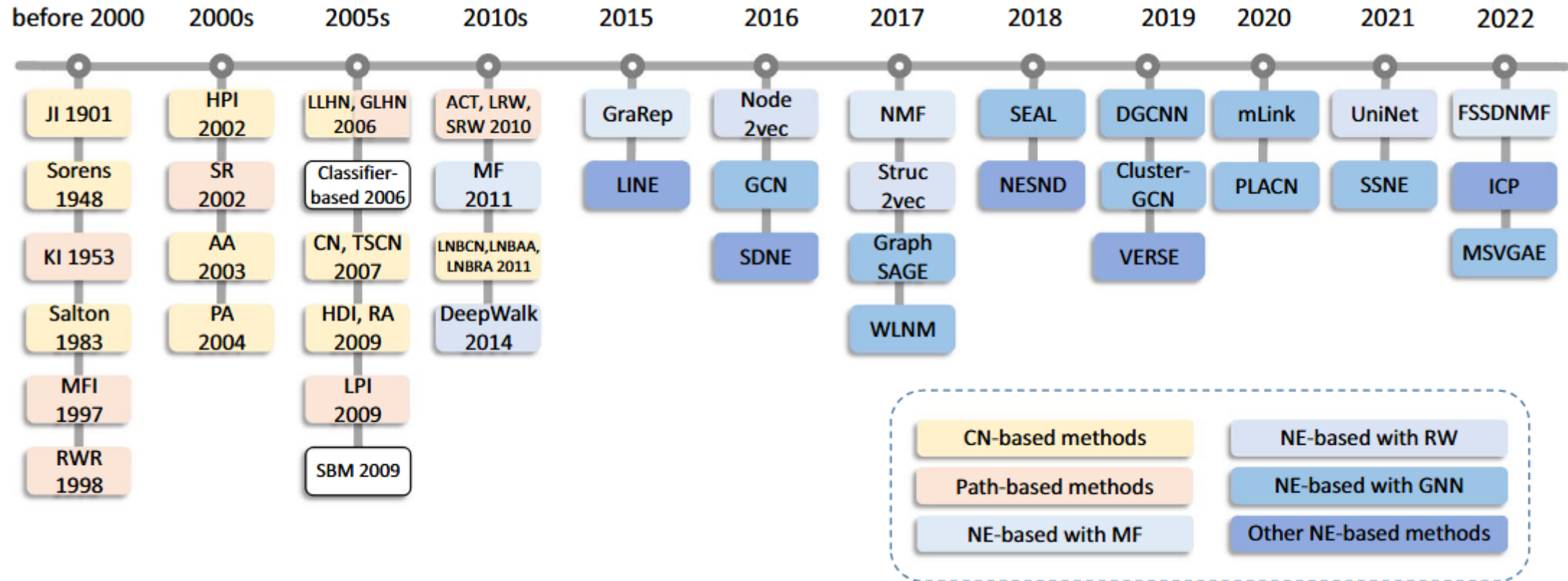
Red nodes are more distant

- Local
 - Common Neighbours (CN)
 - Jaccard's Coefficient (JC)
 - Adamic-Adar (AA)
 - Preferential attachment (PA)
 - Other:
 - Salton index
 - Sorensen index
 - Hub Promoted index
 - Hub Depressed Index
- Global
 - Path based
 - Path based Hitting time
 - SimRank



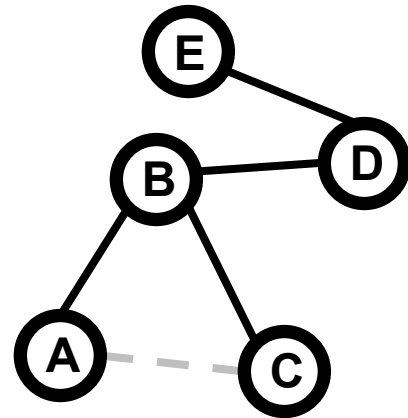


Source: Wu, H., Song, C., Ge, Y., & Ge, T. (2022). Link prediction on complex networks: an experimental survey. *Data science and engineering*, 7(3), 253-278.



Source: Wu, H., Song, C., Ge, Y., & Ge, T. (2022). Link prediction on complex networks: a n experimental survey. *Data science and engineering*, 7(3), 253-278.

- Graph distance: Length of shortest path between two nodes in the graphs
- E.g., the distance between nodes in graphs:



(A, C)	?
(C, D)	?
(A, E)	?

```
# Calculate graph distance
def calculate_graph_distance(G, source, target):
    return nx.shortest_path_length(G, source=source, target=target)

# Instantiate the graph
G = nx.Graph()

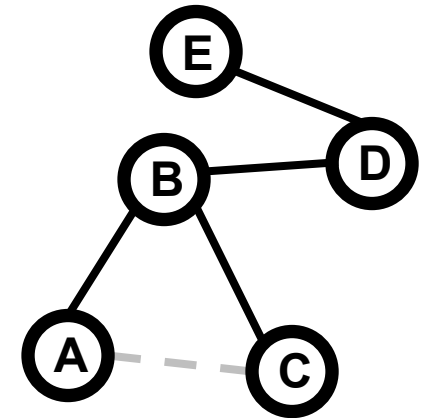
edges = [("A", "B"), ("B", "C"), ("B", "D"), ("D", "E")]
# add node/edge pairs
G.add_edges_from(edges)

# Use spring layout for better visualization
pos = nx.spring_layout(G)
nx.draw(G, pos, with_labels=True)

# Calculate distances
distances = {
    ("A", "C"): calculate_graph_distance(G, 'A', 'C'),
    ("C", "D"): calculate_graph_distance(G, 'C', 'D'),
    ("A", "E"): calculate_graph_distance(G, 'A', 'E'),
}

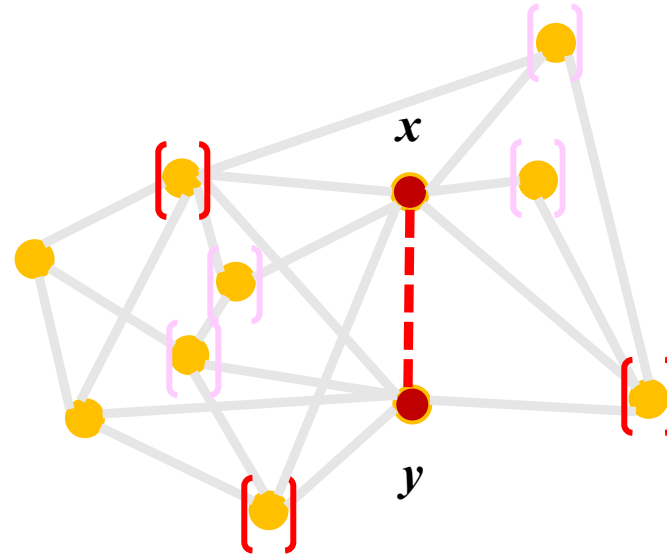
print("Graph distances:")
for nodes, distance in distances.items():
    print(f"{nodes} | {distance}")

### Output
# Graph distances:
# ('A', 'C') | 2
# ('C', 'D') | 2
# ('A', 'E') | 3
```



(A, C)	2
(C, D)	2
(A, E)	3

- How many neighbours are in common between x and y
- x and y have 3 common neighbours, more likely to collaborate
- Let $N(x)$ denote the set of nodes adjacent to x , $N(x) = \{m \mid (x, m) \in E\}$



$$CN = |N(x) \cap N(y)| = 3$$

```
# Instantiate the graph
G = nx.Graph()

# Define edges
edges = [(0, 1), (1, 2), (0, 3), (1, 3), (0, 4), (1, 5), (4, 5), (1, 6),
         (2, 6), (5, 6), (6, 7), (3, 8), (4, 8), (6, 8), (3, 9), (4, 9),
         (1, 9), (8, 9), (2, 10), (6, 10), (7, 10), (9, 10)]

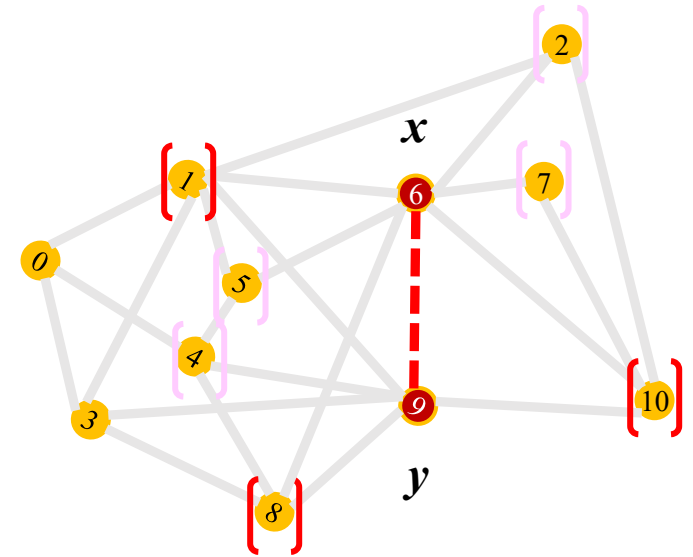
# Add edges to the graph
G.add_edges_from(edges)

# Use spring layout for better visualization
pos = nx.spring_layout(G)
nx.draw(G, pos, with_labels=True)

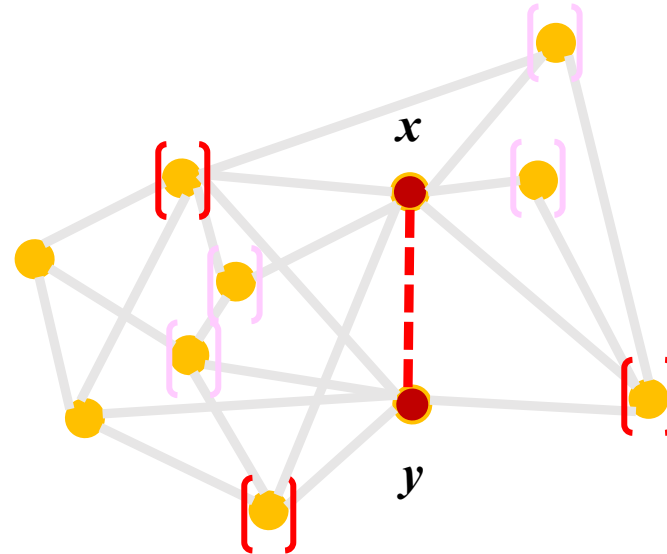
# Calculate common neighbors of nodes 6 and 9
cn_list = sorted(nx.common_neighbors(G, 6, 9))

# Print common neighbors
print(f"The common neighbors of nodes 6 and 9 are: {cn_list}")

### Output
# The common neighbors of nodes 6 and 9 are: [1, 8, 10]
```



- How likely a neighbour of x is also a neighbour of y
- Same as common neighbors, adjusted for degree



$$JC = \frac{|N(x) \cap N(y)|}{|N(x) \cup N(y)|} = \frac{CN}{d_x + d_y - CN}$$

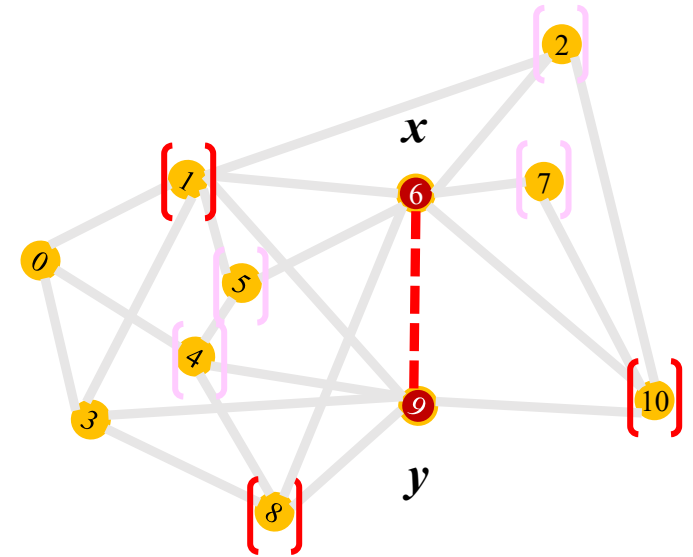


```
# Define list of node pairs
node_list = [(6, 9), (1, 3), (3, 9)]

# Calculate Jaccard coefficients for each pair
coefficients = {}
preds = nx.jaccard_coefficient(G, node_list)
for u, v, p in preds:
    coefficients[(u, v)] = p

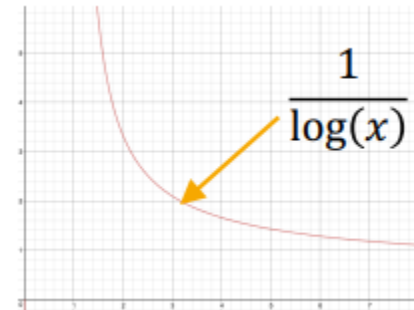
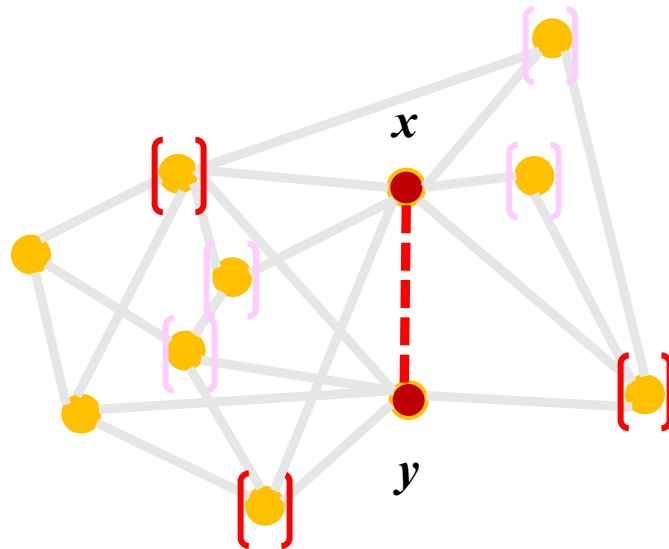
# Print Jaccard coefficients
for nodes, coeff in coefficients.items():
    print(f"The Jaccard coefficient of nodes {nodes} is: {coeff:.8f}")

### Output
# The Jaccard coefficient of nodes (6, 9) is: 0.37500000
# The Jaccard coefficient of nodes (1, 3) is: 0.25000000
# The Jaccard coefficient of nodes (3, 9) is: 0.28571429
```



- Large weight to common neighbours with low degree (the lower the degree the higher the relevance)
- E.g., Neighbours who are linked with 2 nodes are assigned weight = $1/\log(2) = 1.4$
 - Neighbours who are linked with 5 nodes are assigned weight = $1/\log(5) = 0.62$

$$AA = \sum_{z \in CN} \frac{1}{\log d_z}$$



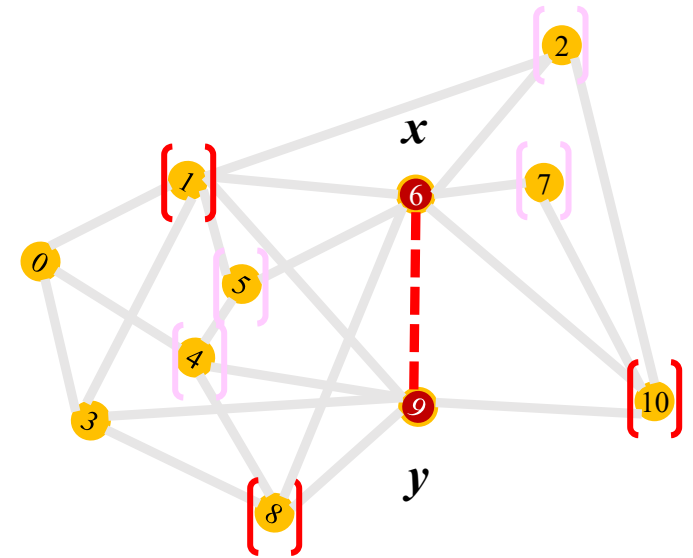
```
# Define list of node pairs
node_list = [(6, 9), (1, 3), (3, 9)]

# Calculate Adamic-Adar indices for each pair
indices = {}
preds = nx.adamic_adar_index(G, node_list)
for u, v, p in preds:
    indices[(u, v)] = p

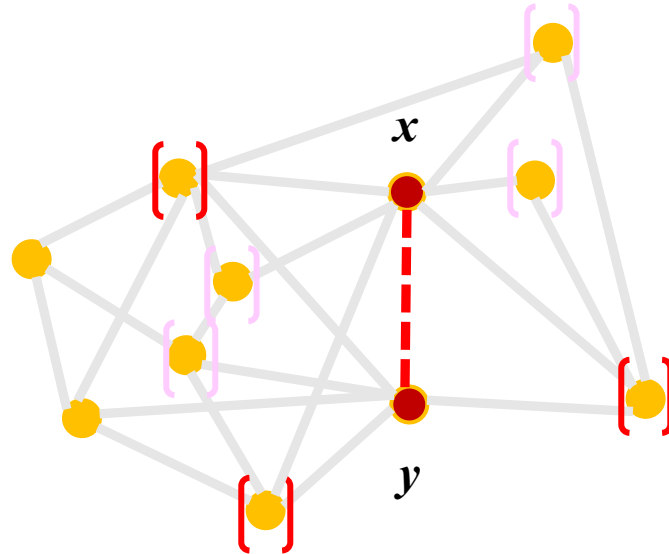
# Print Adamic-Adar indices
for nodes, index in indices.items():
    print(f"The Adamic-Adar index of nodes {nodes} is: {index:.8f}")
```

✓ 0.0s

The Adamic-Adar index of nodes (6, 9) is: 2.00080567
The Adamic-Adar index of nodes (1, 3) is: 1.53157416
The Adamic-Adar index of nodes (3, 9) is: 1.27945815



- Better connected nodes are more likely to form more links.
- The more popular a node is the more probable it will form a link with popular nodes.
- This depends on the degrees of the nodes not on their neighbourhoods



$$PA = |N(x)| \cdot |N(y)| = d_x \cdot d_y$$

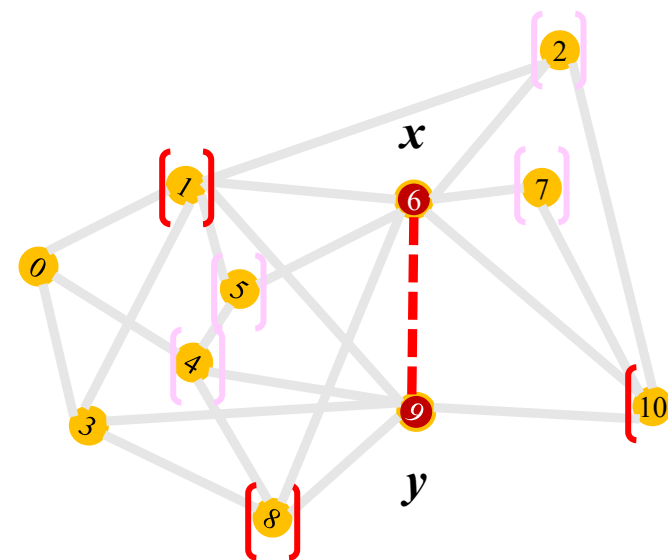
```
# Define list of node pairs
node_list = [(6, 9), (1, 3), (3, 9)]

# Calculate Preferential Attachment scores for each pair
scores = {}
preds = nx.preferential_attachment(G, node_list)
for u, v, p in preds:
    scores[(u, v)] = p

# Print Preferential Attachment scores
for nodes, score in scores.items():
    print(f"The Preferential Attachment score of nodes {nodes} is: {score:.8f}")
```

✓ 0.0s

The Preferential Attachment score of nodes (6, 9) is: 30.00000000
The Preferential Attachment score of nodes (1, 3) is: 24.00000000
The Preferential Attachment score of nodes (3, 9) is: 20.00000000



➤ Salton index

$$score(x, y) = \frac{|N(x) \cap N(y)|}{\sqrt{|N(x)||N(y)|}}$$

➤ Sorensen index

$$score(x, y) = \frac{2|N(x) \cap N(y)|}{|N(x)| + |N(y)|}$$

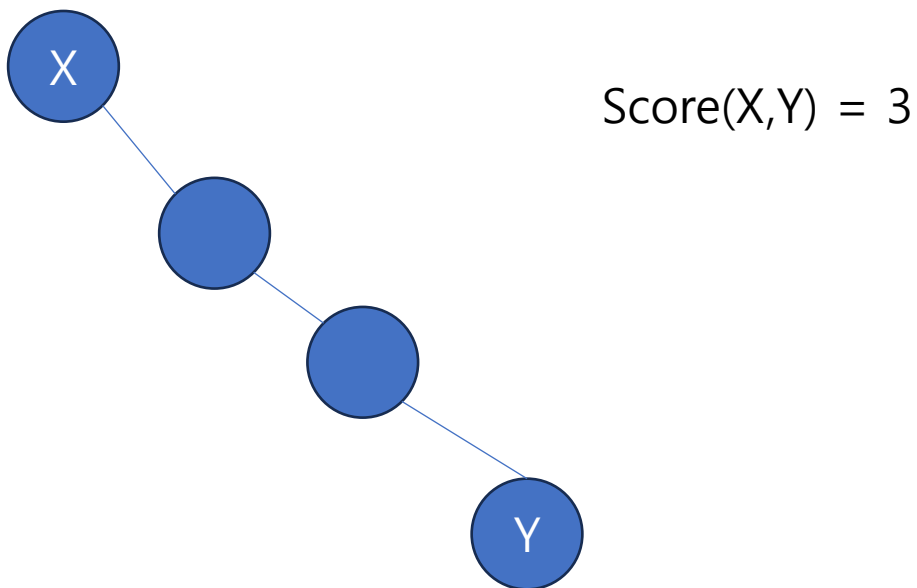
➤ Hub Promoted index

$$score(x, y) = \frac{|N(x) \cap N(y)|}{\min\{|N(x)|, |N(y)|\}}$$

➤ Hub Depressed Index

$$score(x, y) = \frac{|N(x) \cap N(y)|}{\max\{|N(x)|, |N(y)|\}}$$

- Use the (shortest) distance between two nodes as a link prediction measure
- $\text{Score}(x,y)$ = length of shortest path between x and y .
- Very basic approach, it does not consider connections among (x,y) but only the distance



➤ Katz index:

Element (x, y) in the adjacency matrix

$$score(x, y) = \sum_{l=1}^{\infty} \beta^l |paths_{xy}^{(l)}| = \beta A_{xy} + \beta^2 A_{xy}^2 + \dots$$

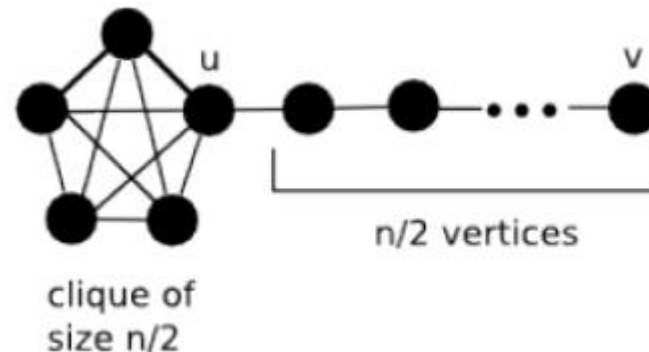
- Sum over ALL paths of length ℓ
- $0 < \beta < 1$ is a parameter of the predictor, exponentially damped to count short paths more heavily
- Small damped parameter = predictions much like common neighbours

- Consider a random walk on graph G that starts at x and iteratively moves to a neighbour of x chosen uniformly random from $N(x)$
- **Hitting time (H_{xy})** from x to y is the expected number of steps it takes for the random walk starting at x to reach y .

$$score(x, y) = -H_{x,y} = -\frac{1}{|N(x)|} \sum_k (1 + H_{k,y})$$

$$H(i, j) = 1 + \sum_{k \sim i} p_{ik} H(k, j), \quad j \neq i, \quad H(i, i) = 0.$$

- Is Hitting Time Symmetric?
 - NOT symmetric
 - E.g., path from u to v is different
From v to u



- Intuition: Two objects are similar, if they are related to similar objects
- Two objects x and y are similar, if they are related to objects a and b respectively and a and b are themselves similar

$$\text{similarity}(x, y) = \frac{\sum_{a \in N(x)} \sum_{b \in N(y)} \text{similarity}(a, b)}{|N(x)| \cdot |N(y)|}$$

- Expresses the average similarity between neighbours of x and neighbours of y :
 $\text{score}(x, y) = \text{similarity}(x, y)$

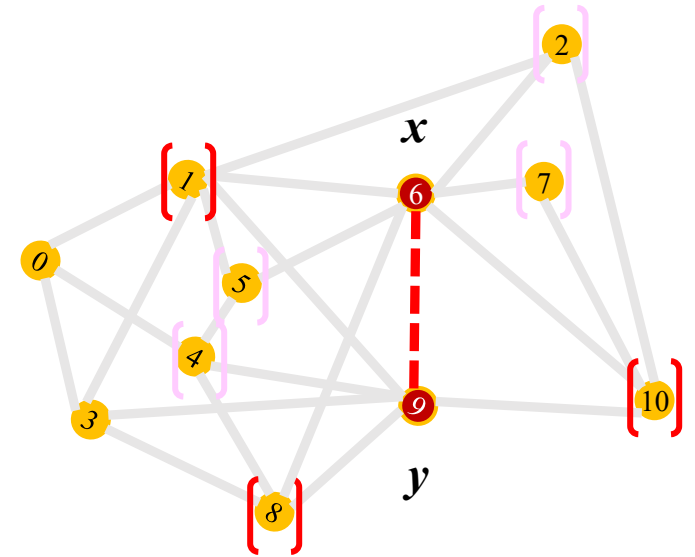
```
# Define source and target nodes
source = 6
target = 9

# Calculate SimRank similarity between source and target
similarity = nx.simrank_similarity(6, source, target)

# Print SimRank similarity
print(f"The SimRank similarity between nodes {source} and {target} is: {similarity}")
```

✓ 0.0s

The SimRank similarity between nodes 6 and 9 is: 0.4283123305678891



- Evaluating link prediction methods in Graph Neural Networks (GNNs) requires measuring results from multiple perspectives. This presentation covers three key metrics used to assess performance.
- Key Metrics
 - **AUC (Area Under the ROC Curve)**: Measures method quality from an overall, global level.
 - **MRR (Mean Reciprocal Rank)**: Focuses strictly on the ranking quality of the predicted edges.
 - **HR@K (Hit Ratio@K)**: Considers the probability of an edge existing within the top K ranked positions.

Area Under the ROC Curve (AUC)

➤ Concept & Advantages

- The most commonly used metric to assess link prediction.
- Highly suitable for datasets with imbalanced distributions (where existing edges are significantly fewer than absent edges).
- Unaffected by class distribution imbalances.

➤ Testing Mechanism

1. Select one existing edge randomly from the test set.
2. Select one non-existent edge randomly.
3. Compare their prediction scores for t total comparisons.
 - If the existing edge score is greater, add 1 to t_1 .
 - If the scores are equal, add 1 to t_2 .

Formula

$$AUC = \frac{t_1 + 0.5t_2}{t}$$

Mean Reciprocal Rank (MRR)

➤ Concept

- Originates from search algorithms and is used to evaluate ranking systems.
- Focuses heavily on how high the correct edges are placed in a ranked list of predictions.

➤ Scoring Mechanism

- If the correct edge is ranked 1st: Score adds 1
- If the correct edge is ranked 2nd: Score adds 0.5
- If the correct edge is ranked n -th: Score adds $1/n$
- If there is no match in the results: Score adds 0

➤ Formula

(For a set of queries Q , where $rank_i$ is the rank position of the first relevant prediction)

$$MRR = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{rank_i}$$

Hit Ratio@K (HR@K)

➤ Concept

- Commonly used to calculate the recall rate in recommendation systems.
- Measures whether the target edge appears within the top K predictions.
- A larger index generally indicates a better performing system.

➤ Variables

- Numerator ($N_{neighbors}$): The number of true neighbors belonging to a given node in the verification set that appear in the first K prediction results.
- Denominator (M_{result}): The total number of true neighbors of a given node in the verification set.

➤ Formula

$$HR@K = \frac{N_{neighbors}}{M_{result}}$$

Category	Tool	Description / Main Use	Programming Support / Interface
General-purpose software	Pajek	Used for general statistics, calculation, and drawing of real or simulated complex networks.	Software tool
	Netdraw		
	Ucinet		
Programming / analysis tool	NetworkX	A Python package for the creation, manipulation, and study of the structure, dynamics, and functions of complex networks.	Python
	igraph	A collection of network analysis tools emphasizing efficiency, portability, and ease of use.	R, Python, Mathematica, C/C++
	statnet	A suite of R packages for the management, exploration, statistical analysis, simulation, and visualization of network data.	R

Category	Tool	Description / Main Use	Programming Support / Interface
Visualization / analysis tool	Gephi	A tool for data analysts and scientists to explore and understand graphs and networks.	Interactive software
	GraphVis	A platform for interactive visual graph mining and relational learning. Also used in the example for EUR network visualization.	Interactive platform
	NodeXL	Excel-based network analysis and visualization tool for collecting, analyzing, and visualizing networks, especially social media networks, in a familiar spreadsheet environment.	Excel add-in / spreadsheet environment
Visualization / multilayer network tool	MuxViz	A platform for visualization and analysis of interconnected multilayer networks, usable as a library or through a browser-based GUI with customizable graphic options.	Library and browser-based GUI

Source: Wu, H., Song, C., Ge, Y., & Ge, T. (2022). Link prediction on complex networks: a n experimental survey. *Data science and engineering*, 7(3), 253-278.

Category	Tool	Description / Main Use	Programming Support / Interface
Programming / visualization toolkit	InfoVis (InfoVis Toolkit)	Interactive graphics toolkit written in Java/Swing for building information visualization applications and components; it supports graph visualizations such as node-link diagrams and adjacency matrices.	Java / Java Swing
Visualization / graph platform	Neo4j	Scalable and performance-oriented; ideal for modeling and querying large-scale graph data.	Cypher query language, GUI tools

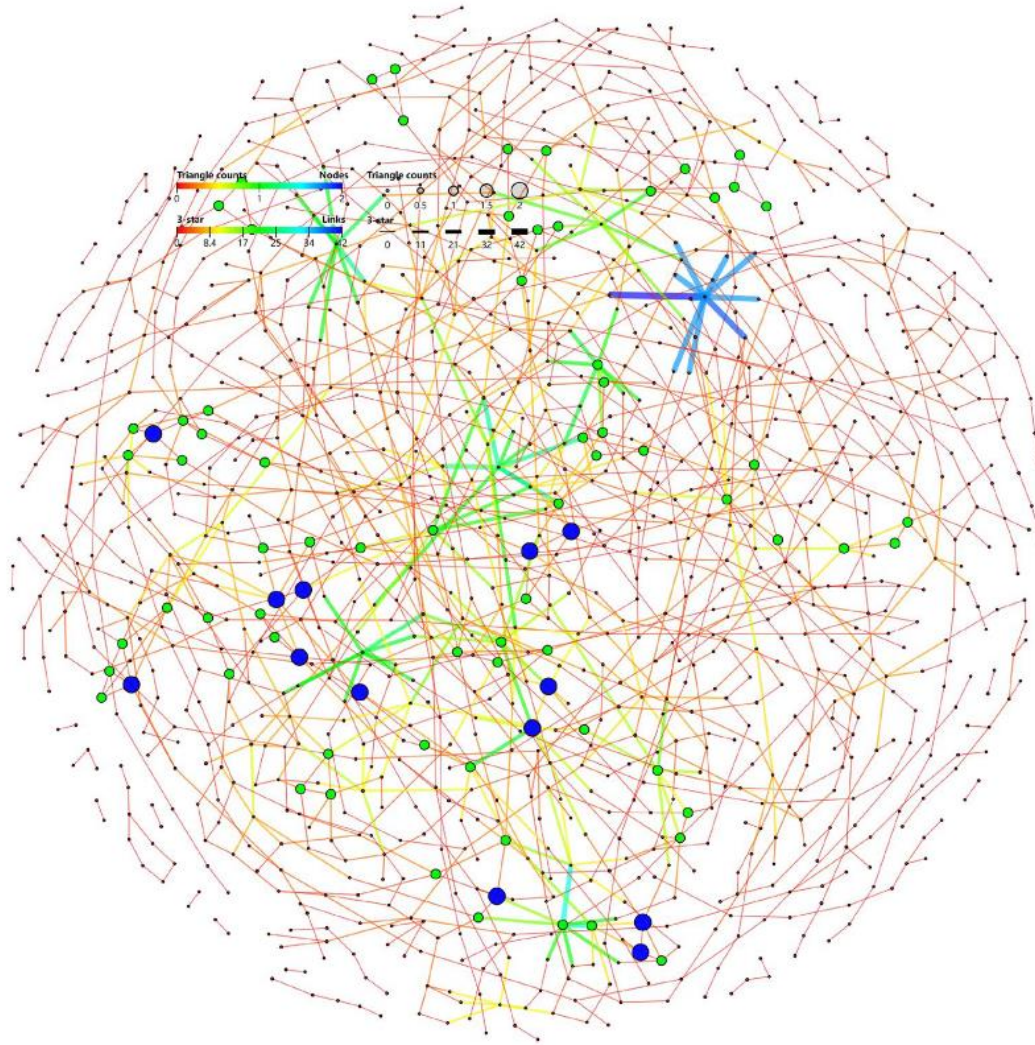
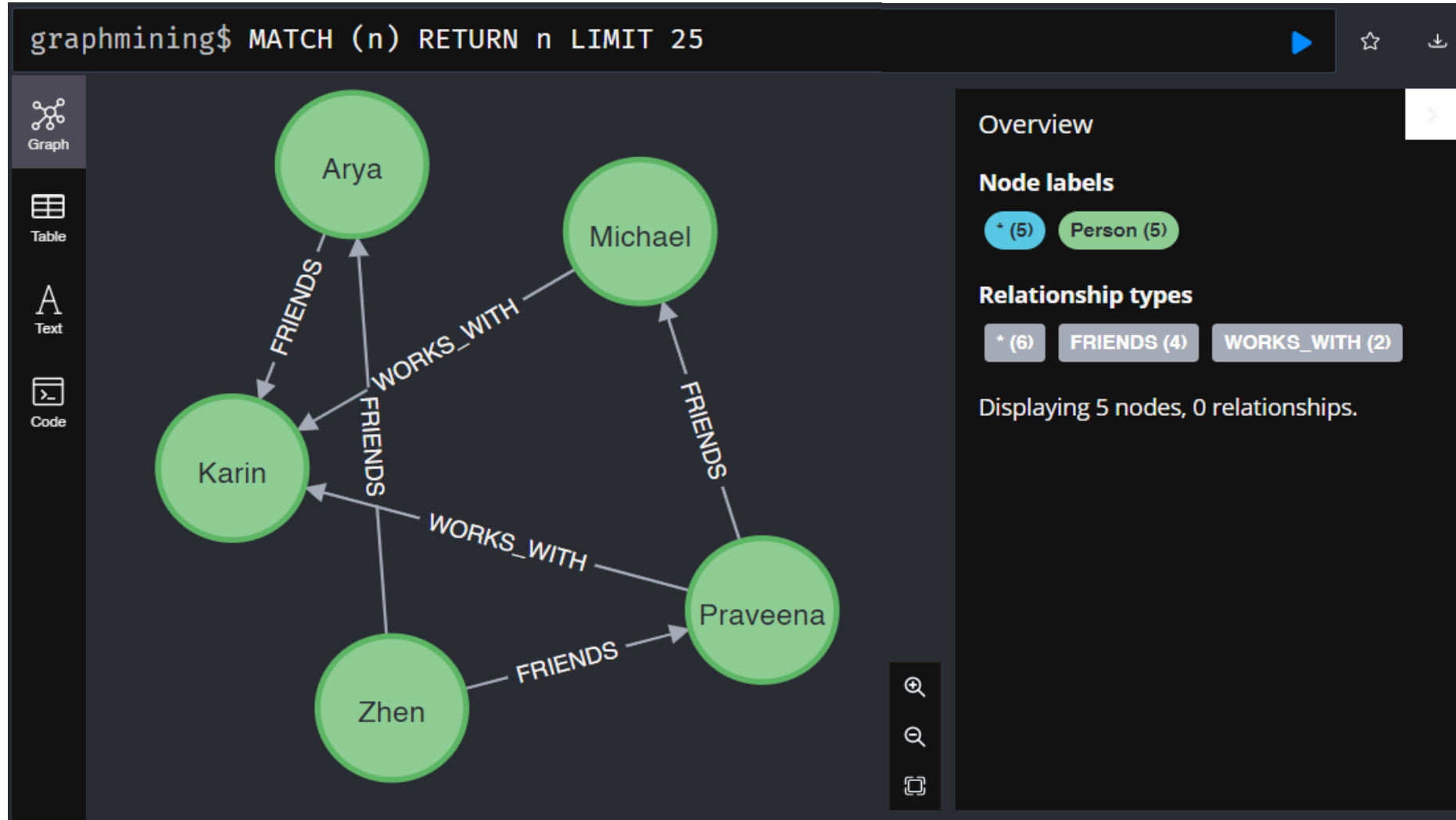


Figure: EUR network visualization using GraphViz

Source: Wu, H., Song, C., Ge, Y., & Ge, T. (2022). Link prediction on complex networks: an experimental survey. *Data science and engineering*, 7(3), 253-278.

➤ Show database



```
1 CREATE
2 (zhen:Person {name: 'Zhen'}),
3 (praveena:Person {name: 'Praveena'}),
4 (michael:Person {name: 'Michael'}),
5 (arya:Person {name: 'Arya'}),
6 (karin:Person {name: 'Karin'}),
7
8 (zhen)-[:FRIENDS]→(arya),
9 (zhen)-[:FRIENDS]→(praveena),
10 (praveena)-[:WORKS_WITH]→(karin),
11 (praveena)-[:FRIENDS]→(michael),
12 (michael)-[:WORKS_WITH]→(karin),
13 (arya)-[:FRIENDS]→(karin)
```



Table



Code

Added 5 labels, created 5 nodes, set 5 properties, created 6 relationships, completed after 21 ms.

Sample code

Use this:

CREATE

// Create Nodes

```
(zhen:Person {name: 'Zhen'}),
(praveena:Person {name: 'Praveena'}),
(michael:Person {name: 'Michael'}),
(arya:Person {name: 'Arya'}),
(karin:Person {name: 'Karin'}),
```

// Create Relationships

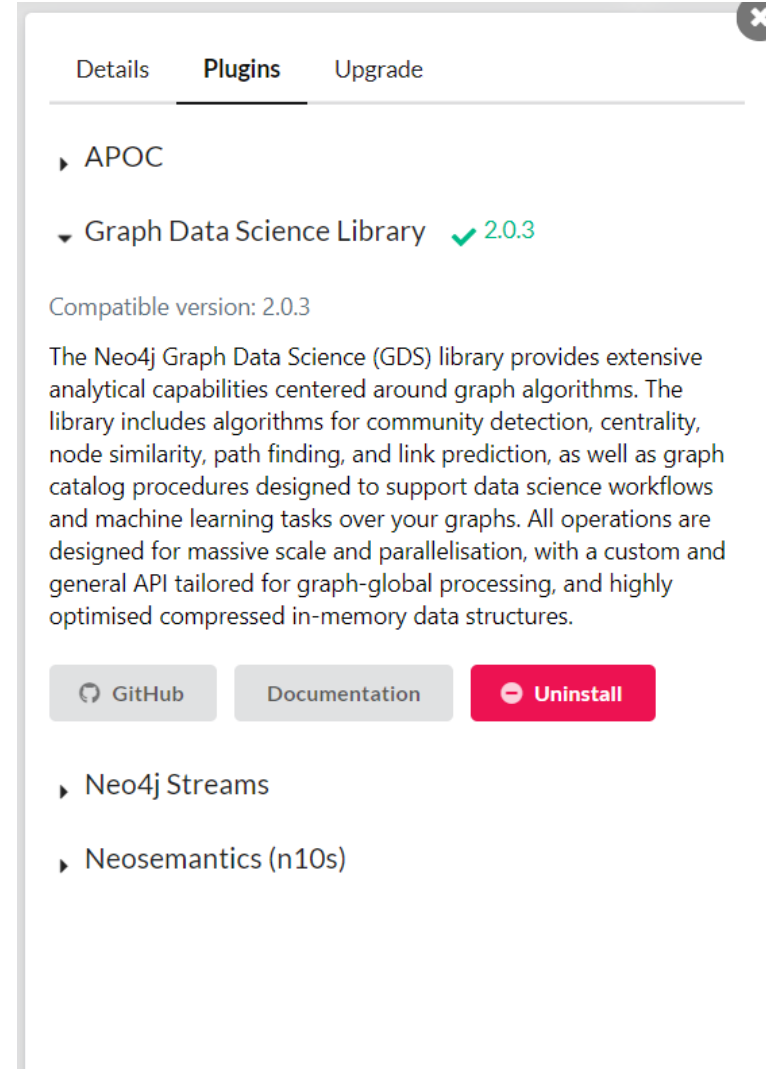
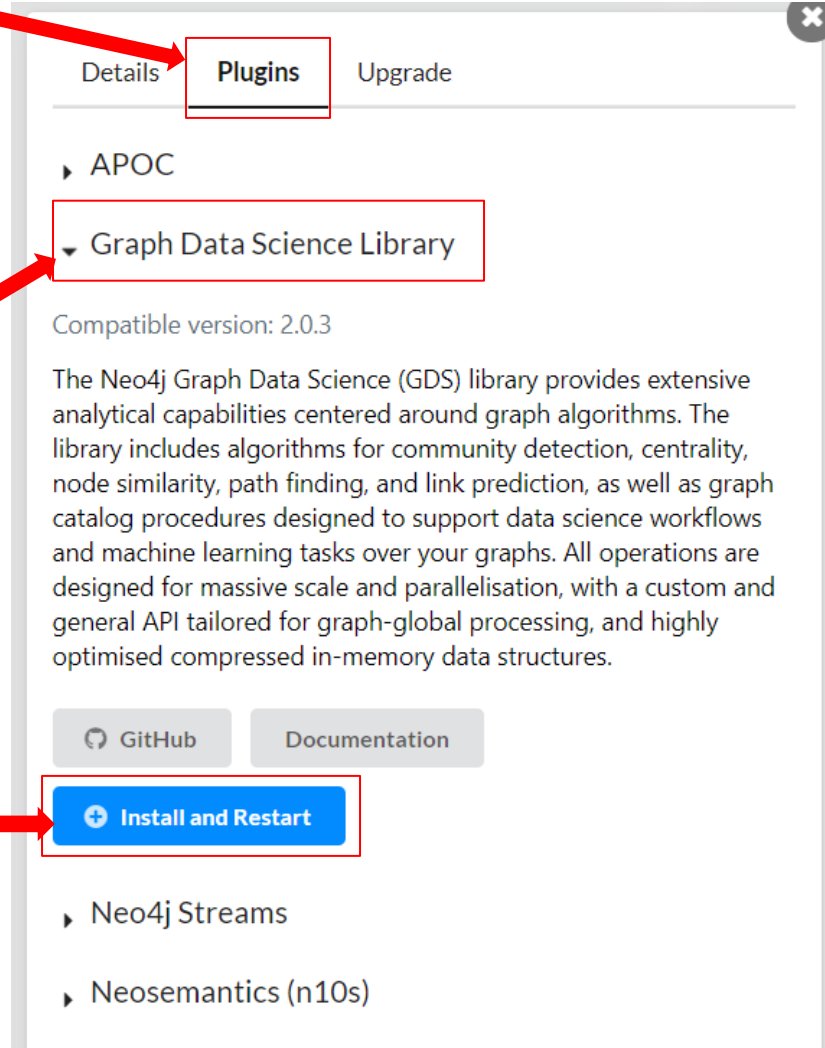
```
(zhen)-[:FRIENDS]->(arya),
(zhen)-[:FRIENDS]->(praveena),
(praveena)-[:WORKS_WITH]->(karin),
(praveena)-[:FRIENDS]->(michael),
(michael)-[:WORKS_WITH]->(karin),
(arya)-[:FRIENDS]->(karin)
```

➤ Install the plugin to use graph algorithms

1. Click "Plugins" tab in the pane on the right.

2. Click on "Graph Data Science Library".

3. Install plugin.



Restart Neo4j to apply graph Data Science Library

- Calculate the number of common neighbors without considering relation type

```
1 MATCH (p1:Person {name: 'Michael'})
2 MATCH (p2:Person {name: 'Karin'})
3 RETURN gds.alpha.linkprediction.commonNeighbors(p1, p2) AS score
```

	score
1	1.0

Use this:

```
MATCH (p1:Person {name: 'Michael'})
MATCH (p2:Person {name: 'Karin'})
RETURN gds.alpha.linkprediction.commonNeighbors(p1, p2) AS score
```

- Calculate the number of common neighbors with considering relation type

```
1 MATCH (p1:Person {name: 'Michael'})
2 MATCH (p2:Person {name: 'Karin'})
3 RETURN gds.alpha.linkprediction.commonNeighbors(p1, p2, {relationshipQuery: "FRIENDS"})
4 AS score
```

	score
1	0.0

Use this:

MATCH (p1:Person {name: 'Michael'})

MATCH (p2:Person {name: 'Karin'})

RETURN gds.alpha.linkprediction.commonNeighbors(p1, p2, {relationshipQuery: 'FRIENDS'})

AS score

- Calculate the Adamic Adar without considering relation type

```
1 MATCH (p1:Person {name: 'Michael'})
2 MATCH (p2:Person {name: 'Karin'})
3 RETURN gds.alpha.linkprediction.adamicAdar(p1, p2) AS score
```

score
0.9102392266268373

Use this:

```
MATCH (p1:Person {name: 'Michael'})
MATCH (p2:Person {name: 'Karin'})
RETURN gds.alpha.linkprediction.adamicAdar(p1, p2) AS score
```


- Calculate the Adamic Adar without considering relation type

```
1 MATCH (p1:Person {name: 'Michael'})
2 MATCH (p2:Person {name: 'Karin'})
3 RETURN gds.alpha.linkprediction.adamicAdar(p1, p2, {relationshipQuery: 'FRIENDS'})
4 AS score
```

Table

Text

Code

"score"
0.0

Use this:

```
MATCH (p1:Person {name: 'Michael'})
```

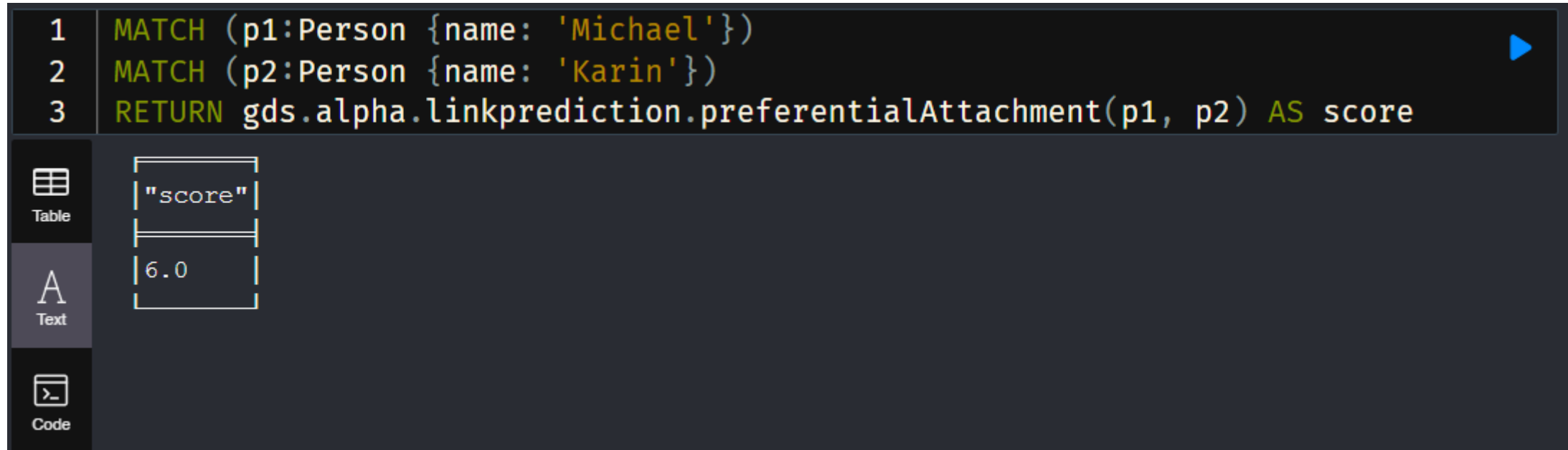
```
MATCH (p2:Person {name: 'Karin'})
```

```
RETURN gds.alpha.linkprediction.adamicAdar(p1, p2, {relationshipQuery: 'FRIENDS'})
```

```
AS score
```

- Calculate the Preferential Attachment without considering the relation type

```
1 MATCH (p1:Person {name: 'Michael'})
2 MATCH (p2:Person {name: 'Karin'})
3 RETURN gds.alpha.linkprediction.preferentialAttachment(p1, p2) AS score
```



The screenshot shows a Neo4j Cypher query interface. The query is: `1 MATCH (p1:Person {name: 'Michael'})`, `2 MATCH (p2:Person {name: 'Karin'})`, `3 RETURN gds.alpha.linkprediction.preferentialAttachment(p1, p2) AS score`. Below the query, there is a result table with one column labeled "score" and one row containing the value 6.0. The interface includes a sidebar with icons for Table, Text, and Code.

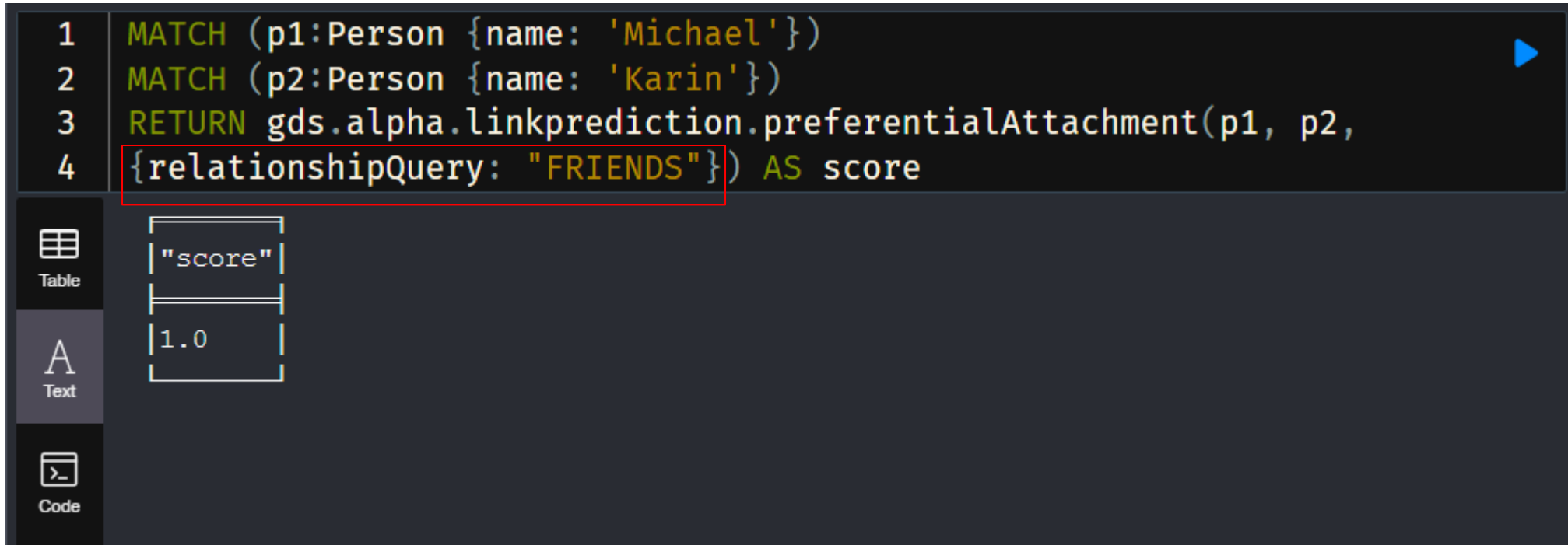
score
6.0

Use this:

```
MATCH (p1:Person {name: 'Michael'})
MATCH (p2:Person {name: 'Karin'})
RETURN gds.alpha.linkprediction.preferentialAttachment(p1, p2)
AS score
```

- Calculate the Preferential Attachment with considering the relation type

```
1 MATCH (p1:Person {name: 'Michael'})
2 MATCH (p2:Person {name: 'Karin'})
3 RETURN gds.alpha.linkprediction.preferentialAttachment(p1, p2,
4 {relationshipQuery: "FRIENDS"}) AS score
```



"score"
1.0

Use this:

```
MATCH (p1:Person {name: 'Michael'})
```

```
MATCH (p2:Person {name: 'Karin'})
```

```
RETURN gds.alpha.linkprediction.preferentialAttachment(p1, p2, {relationshipQuery: "FRIENDS"})
```

```
AS score
```



네트워크 과학연구실
NETWORK SCIENCE LAB



가톨릭대학교
THE CATHOLIC UNIVERSITY OF KOREA

